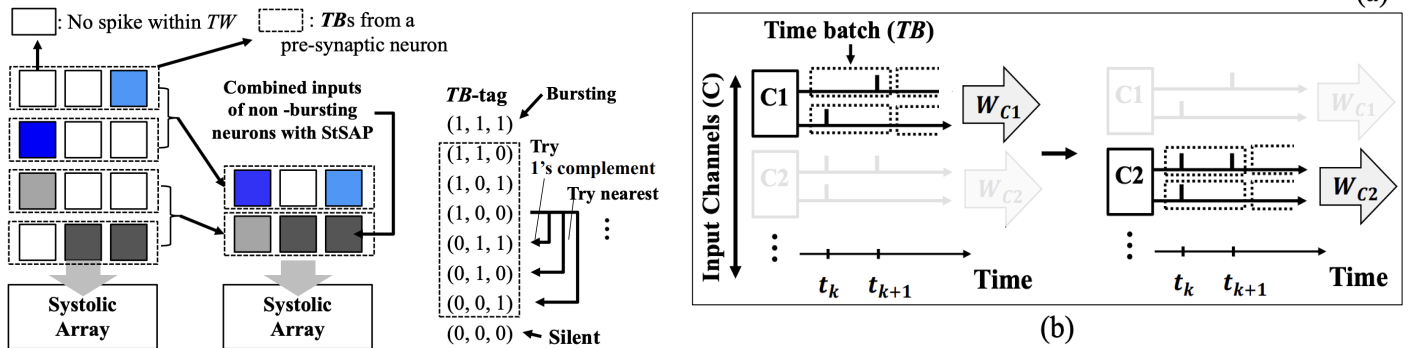


Baseline Comparisons

PTB

Their Architecture



PTB groups multiple consecutive timesteps into a fixed width Time Window (TW) and integrates the synaptic inputs for the whole window in one batched step per PE.

This is mapped onto a 2D systolic array. On top of this, they add StSAP: a scheduling step that finds pairs of neurons whose spike activity **doesn't overlap in time** (ex: 1001, 0110), and merges them so two neurons worth of sparse spikes can share one PE's workload instead of each needing their own.

Membrane update and spike generation still proceed per timestep within the window. The TW size itself is a tunable parameter with real tradeoffs they study explicitly, wider TWs reuse weight data better but waste more act fetch overhead on zero valued activations packed into the window, and their optimal TW and array shape (16x8) is worked out empirically per layer.

The edge our design has

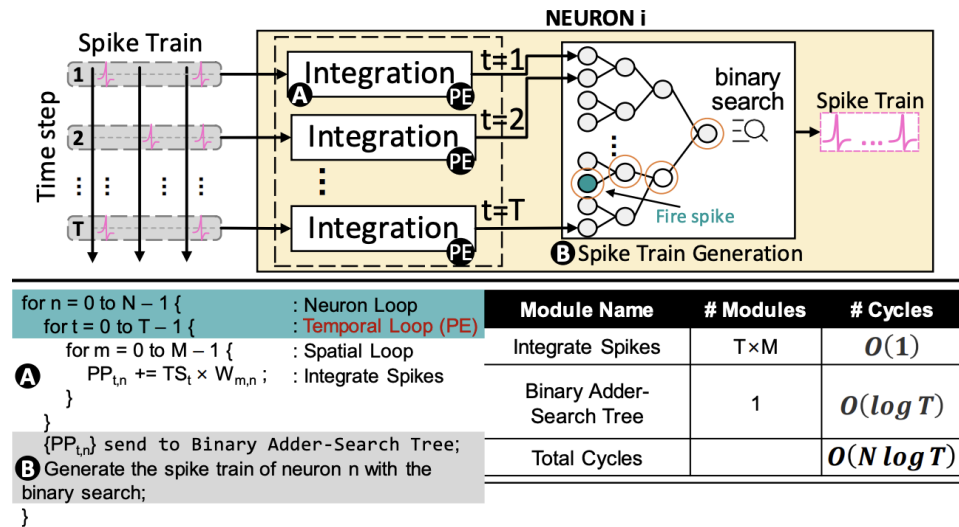
- PTB's TW batching only flattens the sequential timestep cost within one window, it doesn't remove sequential dependence across the full operational period. Our work **parallelizes the full T in one shot**, inside a single PE's accumulator, with no windowing parameter to tune and no multi-pass requirement tied to T
- PTB's sparsity handling (StSAP), it decides which two neurons spikes get processed together on the array, but it doesn't shrink the underlying matrix that gets fetched and stored, the merge is limited to pairs of neurons and happens per time-window, not once offline for the whole operand. Our work's packing **happens once, offline, before the pe array ever sees the data**, and produces an actual smaller matrix (fewer rows/columns) with an index-tag overhead **far below the cost of storing the full dense operand**, a

real reduction in what has to be fetched and stored, not just a reshuffling of scheduling order.

- PTB's weight reuse is temporal, the same weight is reused across the timepoints grouped into one TB, and across PEs in the same row processing different TWs for the same neuron.

In our work the weight is stationary for the entire duration of a tiles matmul, reused against every activation streamed through it, not just within a batching window.

SATO



Their Architecture

- SATO removes the sequential timestep bottleneck** by changing the processing order. It maps **each individual timestep to its own dedicated PE**, allowing all T timesteps to be integrated at the same time in parallel instead of one after another. This eliminates the need to constantly save and read intermediate neuron voltages from memory.
- Those T partial sums then feed into a binary adder-search tree**, which acts as the main hardware engine performing the **Integrate-and-Fire (IF) neuron operations**. The tree sums the values in a parallel hierarchy and uses internal checkpoints to run a fast **binary search**, pinpointing the exact spike times in $O(\log T)$ time instead of checking every step one by one.
- A bucket-sort dispatcher handles spreading the workload evenly** across all these smaller PEs. Because spiking data is highly sparse and unpredictable, the dispatcher groups timesteps together based on their active spike counts. This ensures every PE does an equal amount of math, keeping hardware utilization high and saving power by sharing memory data.

The edge our design has

- SATO uses an IF neuron, which has no leak and therefore retains past inputs indefinitely, potentially **reducing accuracy** compared with LIF. Our LIF stage uses a full leaky with soft reset, unrolled combinationally in one cycle, providing LIF's accuracy benefits **without** added sequential latency.
- SATO's PE to workload mapping is solved by a bucket sort dispatcher balancing spike counts across PEs at runtime. It's a scheduling algorithm reacting to whatever sparsity pattern shows up, with a stated complexity of $O(\text{num of group} * \text{num of PE})$ that **has to run for each new workload**. Our work's tiling and latency are fixed by operand dimensions **known ahead of time**, computed once from a formula rather than solved per inference by a dispatch algorithm.
- SATO doesn't have a packing algorithm that actually shrinks the matrix, its bucket sort dispatcher **only reorders scheduling**, grouping neurons with overlapping non-zero spikes so PEs can share loaded data, but the underlying data volume streamed to the array stays the same size. Our work **packs the matrix itself**, offline, before the array ever runs, producing a genuinely smaller operand, a real compression result, not just a better dispatch order for the same amount of data.